

UPLANDS — Scripts Manual

Operational toolbox for the UPLANDS DEX on the Internet Computer. Use these scripts to bring up a fresh instance, redeploy code, top up cycles, run the simulation, and execute regression tests.

Quick reference

Script	Purpose
<code>master.sh</code>	One-command demo bring-up. Wraps <code>cold_start</code> with sensible defaults.
<code>cold_start.sh</code>	Configurable orchestrator: replica + deploy + top-up + seed + simulator.
<code>seed.sh</code>	Unified seed dispatcher with multiple modes.
<code>inject_history.sh</code>	Fetches CoinGecko hourly prices and injects them as historical trades.
<code>top_up_cycles.sh</code>	Deposits cycles onto a canister's execution balance.
<code>simulate_trading.sh</code>	Background trading simulator with 20 named-trader identities.
<code>seed_exchange.sh</code>	Legacy seed (preserved for backwards compat — superseded by <code>seed.sh full</code>).

master.sh

The friendly default. Zero arguments needed. Use when you want UPLANDS up from any state — replica down, frozen canister, or fresh checkout.

Equivalent to:

```
icp network start
icp deploy
bash scripts/top_up_cycles.sh --amount 100t
bash scripts/seed.sh full --history-days 14 --traders 100
nohup bash scripts/simulate_trading.sh --speed fast &
```

cold_start.sh

The orchestrator. Every flag exposed:

<code>--mode MODE</code>	Seed mode: <code>full</code> <code>empty</code> <code>matching</code> <code>amm</code> <code>pending</code> <code>load</code> (default: <code>full</code>)
<code>--traders N</code>	Number of trader principals (default: 100)
<code>--history-days N</code>	Days of oracle-derived hourly history (default: 14; 0 = disable)
<code>--no-deploy</code>	Skip icp deploy (keep installed wasm)
<code>--no-top-up</code>	Skip the cycle top-up step
<code>--top-up-amount AMT</code>	Cycles to deposit (default: 100t).

	Suffixes k/m/b/t.
--no-seed	Skip the seed step entirely
--no-simulate	Don't start the simulator
--simulate	Force-start the simulator even for test modes

What gets wiped vs preserved

The seed step (specifically `resetExchange`) is destructive. The boundary is precise:

Wiped	Preserved
All open orders	User account balances
All trade history	User profiles + usernames
All AMM pools (and accumulated profit)	Canister cycles
All pending matches	Canister code (deploy is upgrade, not reinstall)
All reserved balances	
Order-book version + level-change log	
Counterparty stats	
Last oracle-aggregate cache	
Pool value history (P&L chart data)	

To redeploy code WITHOUT losing trading state:

```
bash scripts/cold_start.sh --no-seed
```

To top up cycles only:

```
bash scripts/top_up_cycles.sh
```

seed.sh modes

```
bash scripts/seed.sh MODE [--traders N] [--history-days N]
```

Mode	Contents
full	5 named traders + dense books seeded around current oracle prices + AMM pools enabled + 14d oracle history
empty	No-op; verifies markets are initialized. Smoke-test deploys.
matching	2 traders, 1 ask + 1 bid on BTC-ICPUSD at known prices. For matching-engine regression tests.
amm	1 trader + 1 AMM pool on BTC-ICPUSD seeded at <code>refPrice 75000</code> . For AMM regression tests.
pending	Alice places a protected BUY at 75000.0 with a 10s settlement window; Bob has BTC ready to cross. For pending

load N phantom trader principals with balances, sparse book, no AMM. For load testing.

inject_history.sh

```
bash scripts/inject_history.sh [--days N] [--identity NAME]
```

Fetches hourly prices from CoinGecko for BTC, ETH, and ICP, and writes them into the canister as historical trades via the `injectHistoricalTrades` admin endpoint. Uses 5-second pacing between assets and exponential backoff on 429 errors to stay under CoinGecko's free-tier rate limit.

Side effect: writes the latest price per asset to `/tmp/uplands-oracle-prices.txt` for downstream scripts to consume.

top_up_cycles.sh

```
bash scripts/top_up_cycles.sh [OPTIONS]
```

<code>--amount AMT</code>	Cycles to deposit (default: 100t). Suffixes k/m/b/t.
<code>--canister NAME</code>	Canister to top up (default: backend).
<code>--identity ID</code>	Controller identity (default: anonymous).
<code>--mint</code>	On mainnet: mint cycles from ICP first.
<code>--dry-run</code>	Print the planned transfer but don't execute.

Calls `icp canister top-up`, which goes through the management canister's `deposit_cycles` to deposit cycles directly onto the target's execution balance. Handles the case where the canister is frozen (out of cycles) by parsing the principal out of the error message and topping up despite being unable to query the status normally.

Important: `icp cycles transfer` is NOT what you want for canister top-ups. That moves cycles between principals in the cycles ledger (a wallet-style balance) — the canister accepts the transfer but the cycles sit in its ledger account, not its execution balance. Use this script (which uses canister top-up) instead.

simulate_trading.sh

```
bash scripts/simulate_trading.sh \  
  [--reset] [--rounds N] [--speed fast|normal|slow]
```

A continuous trading-loop simulator. Creates 20 named trader identities (`trader_01` . . `trader_20`) on first run, sets balances, then loops placing random limit/market orders, swaps, and cancels at the configured speed.

Bootstraps from the canister's `getMarkets` `lastPrice` on startup (falls back to the oracle snapshot file, then to hardcoded defaults). This is what stops the simulator from repopulating books at stale 2024-vintage prices on restart.

tests/

Regression tests live under `tests/`. Each test sources `tests/_lib.sh`, calls `setup_mode <mode>` to apply a deterministic fixture, and asserts post-conditions.

- **test_matching_engine.sh** — taker BUY crosses one resting ask, exactly one trade, correct remainder.
- **test_amm.sh** — pool seeded at refPrice 75000, after explicit requote has 3 bids + 3 asks.
- **test_pending_match.sh** — protected maker + crossing taker → pending match created, both sides reserved; cancel voids and refunds.
- **test_zombie_free.sh** — fully-pending taker leaves zero 0-qty zombie orders on the book (regression lock).

Run all:

```
bash tests/run_all.sh
```

Common workflows

Cold-start the full demo

```
bash scripts/master.sh
```

Redeploy new code without losing trading state

```
bash scripts/cold_start.sh --no-seed --no-simulate
```

Top up cycles only

```
bash scripts/top_up_cycles.sh
```

Restart simulator with a different speed

```
kill -f simulate_trading.sh
nohup bash scripts/simulate_trading.sh --speed slow \
    > /tmp/uplands-sim.log 2>&1 &
disown
```

Run regression tests

```
bash tests/run_all.sh
```

Recover a frozen canister

```
bash scripts/top_up_cycles.sh
# The script auto-detects out-of-cycles and works around it.
```

Bring up a minimal AMM-test fixture

```
bash scripts/cold_start.sh --mode amm --no-simulate
```